

Mining Software Repositories

- Fonte: «Teaching Mining Software Repositories»
- Autori: Codabux, Fard, Verdecchia, Palomba, Di Nucci, Recupito

Outline

- Introduzione a MSR
- Come progettare uno studio MSR
- Identificare minacce alla validità
- Integrare MSR con altri metodi di ricerca
- Considerare aspetti etici nel mining dei dati

Introduction

- I repository software sono tra le fonti di dati più comuni che consentono l'analisi delle proprietà del software.
- Nel tempo si è esteso il processo di mining anche a: modifiche al software, sforzi collaborativi, aspetti socio-tecnici e dinamiche di processo.

Introduction

- Durante il processo di sviluppo e manutenzione del software, gli sviluppatori utilizzano potenti strumenti che tracciano meticolosamente le informazioni relative al software.
 - robusti sistemi di controllo della versione come Git e Subversion (SVN)
 - strumenti efficienti per la gestione dei problemi come JIRA e Bugzilla,
- Questi tool facilitano la collaborazione e forniscono una tracciabilità completa delle modifiche al codice.

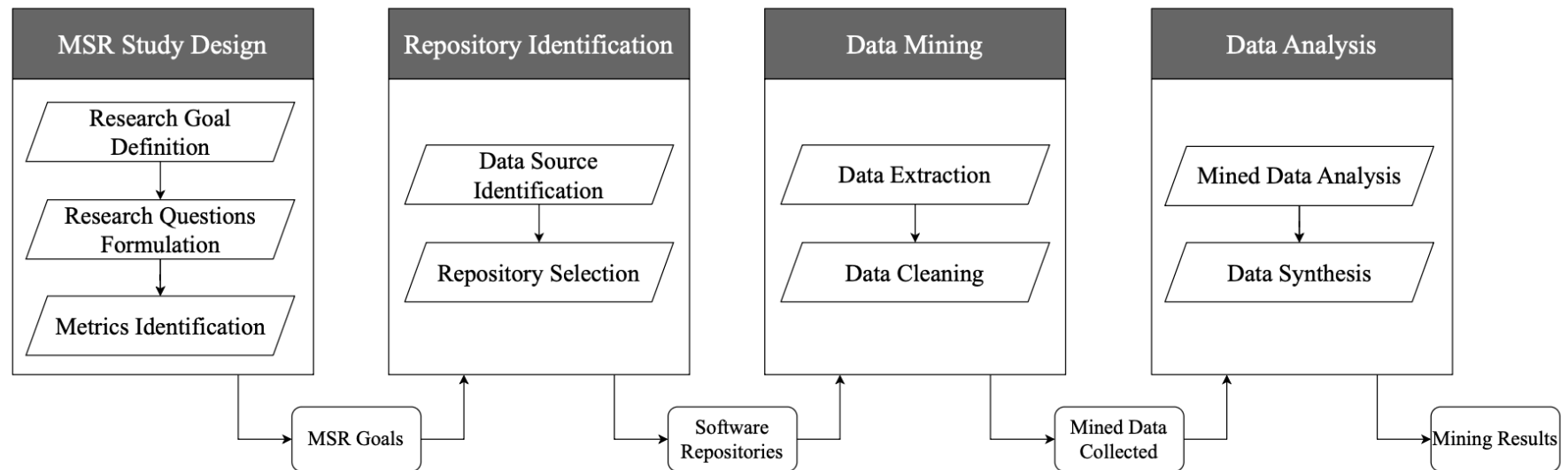
Introduction

- La combinazione di questi strumenti consente agli sviluppatori di acquisire una conoscenza approfondita di tutti gli aspetti chiave del progetto di evoluzione del software,
 - modifiche al software,
 - risoluzione dei problemi,
 - correzione di bug e refactoring strategico.
- MSR è il campo che sfrutta le informazioni contenute nei repository del software per condurre indagini e analisi approfondite, svelando aspetti del processo di sviluppo del software.

Introduction

- Studi nell'ambito dell'ingegneria del software hanno dimostrato le capacità intrinseche dell'analisi dei repository.
- Diversi studi si sono focalizzati sulla possibilità di costruire modelli predittivi per i problemi di qualità del software.
 - I modelli predittivi potrebbero essere costruiti a partire dalle etichette di tracciamento dei problemi per rilevare i bug.
- L'enorme quantità di dati che è possibile estrarre dai repository di codice sorgente ci permette di costruire modelli complessi e utili utilizzando i Large Language Models,
 - strumenti utili che eseguono attività di programmazione relative alla generazione, alla sintesi e all'analisi del codice, producendo anche codice non complesso e altamente manutenibile (ad esempio, Copilot).

Setting up a Source Code Repository Mining Study



Insieme di attività coinvolte nel processo di studio MSR

Research goal formulation

- La formulazione dell'obiettivo di ricerca di uno studio di mining attraverso il paradigma GQM segue una struttura sistematica.

Analyze *defect prediction models*
for the purpose of *defect prediction*
with respect to *risky changes*
from the point of view of *software developers and reviewers*
in the context of *open source and commercial projects from multiple domains.*

(Kamei et al. 2013) Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi. *A large-scale empirical study of just-in-time quality assurance*. IEEE Transactions on Software Engineering, 39(6):757–773, 2013.

Research Questions Definition

- Le domande di ricerca (RQ) devono essere derivate direttamente dall'obiettivo di ricerca formulato nella fase precedente.
- Considerando il potenziale impatto della ricerca, traendo ispirazione da varie fonti e mettendole in relazione con possibili fenomeni e concetti ad essi correlati.
- Una volta generato un elenco completo di potenziali domande di ricerca, il passo finale consiste nel selezionare domande di ricerca specifiche da approfondire.

Research Questions Definition

- *RQ1 : How well can we predict defect-inducing changes?*
- *RQ2 : Does prioritizing changes based on predicted risk reduce review effort?*
- *RQ3 : What are the major characteristics of defect-inducing changes?*

- Possiamo osservare che RQ1 è quella più strettamente correlata all'obiettivo.
- RQ2 e RQ3 vengono utilizzate per sviluppare l'obiettivo della ricerca e fornire ulteriori nozioni complementari sull'argomento considerato.

Research Questions Definition

- Come potenziali suggerimenti, altri esempi di RQ per il goal di esempio potrebbero essere:

- *RQ: Can Large Language Models be used to identify bugs?*
- *RQ: Is there a relation between technical debt and software energy efficiency?*
- *RQ: How does software quality evolve in microservice architectures?*
- *RQ: Which code smells are more frequent in AI-centric software projects?*
- *RQ: What are the most common causes of test flakiness?*

Metrics identification

- La selezione delle metriche dipende strettamente dalle specifiche RQ in questione, che possono essere qualitative, quantitative o un mix di entrambe.
- Le metriche selezionate possono essere atomiche, ad esempio righe di codice o numero di difetti, oppure composite, ad esempio difetti per riga di codice.
- È quindi possibile comporre lo stesso insieme di metriche atomiche in modi diversi per rispondere a diverse RQ.

Homework

- Attraverso l'approccio GQM, definire un obiettivo, formulare tre RQ e identificare cinque metriche (tenendo conto delle RQ) per analizzare quali fattori di sviluppo influenzano la propensione ai bug.

Identifying High-Quality Source Code Repositories

- **Data Sources**

- I repository possono essere di natura statica o dinamica e possono essere classificati come segue:
- Repository storici
 - (i) *repository di bug o difetti*, come Jira e Bugzilla, che aiutano a tracciare e gestire i difetti di un sistema software
 - (ii) *comunicazioni archiviate* come mailing list, e-mail e messaggi di chat, che sono una fonte di discussioni su molteplici aspetti di un sistema software
 - (iii) *repository di controllo del codice sorgente* che tengono traccia di tutte le modifiche apportate agli artefatti del sistema software (ad esempio, codice sorgente, Pull Request, messaggi di commit, documentazione). Git è uno dei repository di controllo del codice sorgente più diffusi

Identifying High-Quality Source Code Repositories

- I repository di codice sorgente controllano e gestiscono i progetti software.
 - Le caratteristiche tipiche includono *l'hosting del codice sorgente*, il *tracciamento dei bug*, le funzionalità di *documentazione*, le *mailing list* e i *forum*. Tra i repository di codice sorgente più noti si annoverano Sourceforge, GitHub e Bitbucket.
- Repository runtime come i log di distribuzione che tengono traccia delle informazioni e delle azioni relative alla distribuzione di un sistema software.
 - I log di distribuzione sono essenziali per monitorare le configurazioni e le fasi delle istanze di distribuzione.

Identifying High-Quality Source Code Repositories

- **Source Code Repositories Selection Criteria**
 - *Linguaggio/i di programmazione*, ad esempio Java per i progetti dell'ecosistema Apache.
 - *Dimensione e complessità del repository*, ad esempio numero di righe di codice (LOC) numero di commit.
 - *Dominio*, ad esempio giochi, visualizzazione, database, parser, test.
 - *Repository attivo o inattivo*, ad esempio ci sono state attività di commit recenti (negli ultimi 6 mesi) sul repository? Quando è stato effettuato l'ultimo commit?
 - *Tipologie di repository*, ovvero repository base (non forkati) o repository forkati.
 - *Scopo del repository*. Non tutti i repository sono repository software. Alcuni sono utilizzati per la sperimentazione, l'hosting di siti web, progetti accademici e personali (non collaborativi).
 - *Posizione del repository*. Alcuni progetti sono ospitati su più piattaforme.
 - *Popolarità del repository*, ad esempio numero di stelle e contributori.

Identifying High-Quality Source Code Repositories

- **Considerations when Selecting Data Sources**
 - È necessario prendere in considerazione diversi fattori per mitigare le *potenziali minacce* quando si valutano i repository di codice sorgente ai fini di condurre studi empirici
 - I repository comprendono *diverse entità* oltre ai progetti software,
 - fungendo da repository per l'archiviazione web gratuita, libri online o repository che ospitano progetti con altre caratteristiche.
 - Numerosi repository presentano *caratteristiche transitorie*,
 - essendo di breve durata, inattivi o dedicati a compiti, progetti studenteschi, obiettivi didattici, uso personale o scopi di archiviazione.
 - Pertanto, se la domanda di ricerca riguarda lo sviluppo di software, questi progetti dovrebbero essere rimossi dal dataset.

Identifying High-Quality Source Code Repositories

- **Considerations when Selecting Data Sources**
 - Inoltre, è fondamentale dare priorità ai repository che ospitano *progetti software strutturati*.
 - Di conseguenza, negli studi relativi allo sviluppo di software, diventa imperativo purificare il dataset da tali repository.
 - Inoltre, molti progetti hanno pochi commit e non tutti utilizzano le pull request, con conseguente distorsione o squilibrio dei dati.
 - Il processo di commit nelle pull request e per le revisioni del codice dipende dalla prassi di GitHub di registrare i commit, che dovrebbe essere anch'essa esaminata attentamente
 - Una strategia per mitigare tali potenziali minacce consiste nell'astenersi dal considerare come committer repository che includono molti utenti non registrati.

Identifying High-Quality Source Code Repositories

- **Considerations when Selecting Data Sources**
 - Inoltre, i progetti che si identificano esplicitamente come mirror nelle loro descrizioni dovrebbero essere attentamente valutati o esclusi.
 - Per uno studio MSR, raccomandiamo di avere un elenco di *criteri di inclusione ed esclusione* basati sugli obiettivi dello studio e sulle domande di ricerca.
 - È possibile condurre *un'esplorazione manuale* su repository campione per valutare se sono necessarie modifiche ai criteri di selezione per garantire la raccolta di un set di dati di alta qualità.

Identifying High-Quality Source Code Repositories

- **Considerations when Selecting Data Sources**

- Nella selezione dei repository, tenere presente che la selezione e la classificazione dei repository in base al numero di stelle potrebbero favorire strategie di marketing attive e non le consolidate pratiche di ingegneria del software.
 - *basarsi esclusivamente sul numero di stelle può compromettere la validità dei dati raccolti.*
- Un altro aspetto da considerare è la correttezza delle euristiche e dei termini chiave utilizzati per la selezione dei repository. Queste euristiche dovrebbero essere esaminate attentamente
 - I criteri per la scelta delle fonti di dati possono portare ad una raccolta di repository rumorosa e, di conseguenza, a un dataset distorto. Indagini approfondite rivelano se questa distorsione è correlata a metriche imprecise per la selezione dei repository o è dovuta alla natura dei dati.

Identifying High-Quality Source Code Repositories

- **Considerations when Selecting Data Sources**
 - Comprendere i *limiti dei dati dei repository* è un altro aspetto da considerare, poiché i dati dei repository *non possono portare a conclusioni causali e possono solo mostrare correlazioni*.
 - Inoltre, i progetti attivi potrebbero non includere tutte le loro attività di sviluppo in GitHub. Pertanto, è opportuno considerare altre risorse.
 - In generale, i *risultati delle MSR devono essere considerati nel contesto in cui gli studi vengono condotti*, il che è fondamentale per rivelare la causa effettiva di particolari conclusioni.
 - L'utilizzo dei repository potrebbe variare da un progetto all'altro.
 - Pertanto, i ricercatori dovrebbero *esaminare attentamente gli aspetti socio-tecnici* per comprendere meglio l'utilizzo dei repository prima di giungere a delle conclusioni.

Homework

- Selezionare attentamente le fonti prima di analizzarle.
- Nella selezione delle fonti di dati, seguire le considerazioni date per un'estrazione efficace dei dati, essenziale per:
 - rispondere alle domande di ricerca
 - prevenire interpretazioni errate
 - garantire l'integrità dello studio

Homework

- Selezionare tre progetti GitHub concentrandosi su tre criteri di inclusione ed esclusione, ad esempio
 - progetti scritti in diversi linguaggi di programmazione,
 - di diversi domini e
 - progetti attivi con almeno 200 commit nell'ultimo anno come criteri di inclusione.
- I criteri di esclusione potrebbero includere
 - progetti accademici
 - e personali
 - e progetti ospitati su più piattaforme.
- ~~Una volta selezionati i progetti, i dati di tracciamento dei problemi possono essere analizzati ulteriormente.~~

Data Extraction

- L'estrazione dei dati inizia una volta selezionato un repository di codice sorgente e dipende dagli obiettivi dello studio
 - Negli studi di analisi predittiva, questa fase prevede l'estrazione di dati relativi a variabili indipendenti e dipendenti.
 - L'obiettivo è estrarre dati relativi alla prima variabile in grado di prevedere accuratamente la prima variabile in futuro.
 - Ad esempio, consideriamo la previsione dei difetti just-in-time (**Kamei et al. 2013**), che mira a prevedere i *commit difettosi* in base alle loro caratteristiche e prevede l'estrazione di dati dai commit per estrarre informazioni utili a prevedere tali commit difettosi. Tali commit possono essere analizzati in sequenza, oppure è possibile analizzare commit specifici in una determinata finestra temporale.

Data Extraction

- La variabile dipendente che dovremo estrarre è la propensione al fallimento dei nostri commit.
- Le variabili indipendenti sono le caratteristiche che dovrebbero portare a tale fallimento.
- È essenziale considerare che i concetti descritti sono generalizzabili
 - per prevedere difetti,
 - code smell
 - vulnerabilità di sicurezza.

Data Extraction

- Nella previsione just-in-time dei difetti, il primo passo consiste nell'identificare i commit che risolvono i difetti.
 - è necessario raccogliere i problemi chiusi e correlati ai bug (ad esempio, con le etichette bug e bugfix).
- I repository di codice sorgente come GitHub forniscono sistemi di tracciamento dei problemi che collegano le segnalazioni di problemi ai commit di correzione dei bug.
 - Un messaggio di commit viene etichettato come correttivo di un difetto se corrisponde a un'espressione regolare come la seguente.

(bug|fix|error|crash|problem|fail|defect|patch)

- Vengono mantenuti solo i commit che modificano almeno un file del codice sorgente.

Data Extraction

- Vengono analizzati i commit dal più recente al più vecchio.
 - Per ogni file corretto, l'algoritmo SZZ [**SZZ5**] identifica automaticamente il commit più vecchio che ha modificato le righe di codice coinvolte nella correzione.
 - È importante notare che SZZ si è evoluto nel corso degli anni e sono disponibili diverse varianti.
- Una volta individuato il commit che ha introdotto il difetto, tutti i commit compresi tra il commit che ha introdotto il difetto (incluso) e il commit che lo ha corretto (escluso) vengono etichettati come soggetti a errori.
- Dopo aver determinato la propensione agli errori dei componenti del codice, è possibile estrarre metriche in grado di prevedere tali errori.
 - Nell'esempio, **Kamei et al.** hanno estratto *14 metriche* raggruppate in cinque dimensioni, ovvero *diffusione, dimensione, scopo, cronologia ed esperienza*.
 - È fondamentale considerare che metriche diverse si applicano in contesti diversi.
 - Le metriche *strutturali* (**Chidamber & Kemerer**) si concentrano sulle proprietà strutturali estratte tramite l'analisi del codice sorgente.
 - Le metriche *delta* [dBRBvD19] rilevano la quantità di modifiche apportate a un file tra due versioni successive.
 - Le metriche di *processo* [MPS08] considerano gli aspetti relativi al processo di sviluppo piuttosto che al codice stesso.
- Sono disponibili diversi tool per analizzare i repository di codice sorgente ed estrarre metriche preziose.
 - PyDriller [**SAB18**] è un framework Python che consente l'estrazione di informazioni da qualsiasi repository Git, come commit, sviluppatori, modifiche, differenze e codice sorgente, ed esporta rapidamente i dati in file CSV.

[**SAB18**] Davide Spadini, Maurício Aniche, and Alberto Bacchelli. *Pydriller: Python framework for mining software repositories*. In Proceedings of the 2018 26th ACM Joint meeting on european software engineering conference and symposium on the foundations of software engineering, pages 908–911, 2018.

[**SZZ5**] Acek Sliwerski, Thomas Zimmermann, and Andreas Zeller. When do changes induce fixes? ACM sigsoft software engineering notes, 30(4):1–5, 2005.

Data Extraction

- *Una volta individuato il commit che ha introdotto il difetto, tutti i commit compresi tra il commit che ha introdotto il difetto (incluso) e il commit che lo ha corretto (escluso) vengono etichettati come soggetti a errori.*

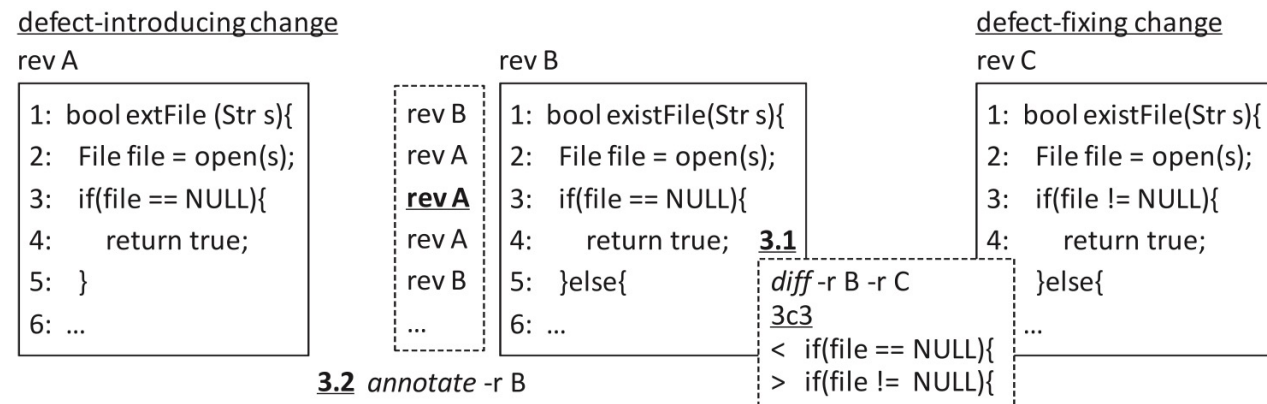


Fig. 1. Example of defect-fixing and defect-introducing changes [23].

Confronto tra la revisione del file dopo la correzione del difetto (rev. C) e la revisione del file prima della correzione del difetto (rev. B) utilizzando il comando diff (3.1) per ottenere le righe effettivamente modificate dalla correzione del difetto.

Si identifica quindi la revisione più recente (rev. A) in cui è stata modificata la riga #3 della rev. B utilizzando il comando annotate (3.2) e si etichetta la modifica della revisione identificata come la modifica che ha introdotto il difetto.

Data Extraction

- Kamei et al. hanno estratto *14 metriche* raggruppate in cinque dimensioni, ovvero *diffusione, dimensione, scopo, cronologia ed esperienza*.

Dim.	Name	Definition	Rationale	Related Work
Diffusion	NS	Number of modified subsystems	Changes modifying many subsystems are more likely to be defect-prone.	The defect probability of a change increases with the number of modified subsystems [41].
	ND	Number of modified directories	Changes that modify many directories are more likely to be defect-prone.	The higher the number of modified directories, the higher the chance that a change will induce a defect [41].
	NF	Number of modified files	Changes touching many files are more likely to be defect-prone.	The number of classes in a module is a good feature of post-release defects of a module [47]
	Entropy	Distribution of modified code across each file	Changes with high entropy are more likely to be defect-prone, because a developer will have to recall and track large numbers of scattered changes across each file.	Scattered changes are more likely to introduce defects [10], [19].
Size	LA	Lines of code added	The more lines of code added, the more likely a defect is introduced.	Relative code churn measures are good indicators of defect modules [43], [46].
	LD	Lines of code deleted	The more lines of code deleted, the higher the chance of a defect.	
	LT	Lines of code in a file before the change	The larger a file, the more likely a change might introduce a defect.	Larger modules contribute more defects [27].
Purpose	FIX	Whether or not the change is a defect fix	Fixing a defect means that an error was made in an earlier implementation, therefore it may indicate an area where errors are more likely.	Changes that fix defects are more likely to introduce defects than changes that implement new functionality [16][52].
History	NDEV	The number of developers that changed the modified files	The larger the NDEV, the more likely a defect is introduced, because files revised by many developers often contain different design thoughts and coding styles.	Files previously touched by more developers contain more defects [32].
	AGE	The average time interval between the last and the current change	The lower the AGE (i.e., the more recent the last change), the more likely a defect will be introduced.	More recent changes contribute more defects than older changes [15].
	NUC	The number of unique changes to the modified files	The larger the NUC, the more likely a defect is introduced, because a developer will have to recall and track many previous changes.	The larger the spread of modified files, the higher the complexity [10], [19].
Experience	EXP	Developer experience	More experienced developers are less likely to introduce a defect.	Programmer experience significantly decreases the defect probability [41].
	REXP	Recent developer experience	A developer that has often modified the files in recent months is less likely to introduce a defect, because she will be more familiar with the recent developments in the system.	
	SEXP	Developer experience on a subsystem	Developers that are familiar with the subsystems modified by a change are less likely to introduce a defect.	

Data Extraction

- **Data Cleaning**

- tecniche e procedure per garantire che i set di dati siano accurati, coerenti e affidabili
- Quando si estraggono dati dai commit, il primo problema riguarda l'estrazione dei dati da *commit complessi*.
 - Questo tipo di commit contiene più elementi informativi relativi a modifiche diverse, spesso causate dall'introduzione di *commit di modifica multipli* per la correzione di bug.
 - Queste modifiche possono creare problemi durante l'analisi del corrispondente archivio di versione, introducendo imprecisioni. Ad esempio, se viene apportata una modifica complessa per correggere un bug, tutti i file ad essa associati potrebbero essere erroneamente etichettati come difettosi nel contesto storico.
 - Quando si trattano questi tipi di commit, si consiglia di comprendere le modifiche all'interno di un commit e di considerarlo separatamente come *partizioni di un insieme di modifiche*.

Data Extraction

- **Data Cleaning**

- Un ulteriore problema critico è l'inclusione dei *commit di merge*.
 - Sforzo aggiuntivo poiché molte operazioni di merge potrebbero non essere etichettate come tali, portando all'ispezione manuale del contenuto delle modifiche per identificarle.
- Altri tipi di modifiche e commit che possono fuorviare l'analisi: *commit volti a implementare modifiche omesse nel commit precedente*.
 - La decisione di includere o escludere questi commit può influenzare significativamente la complessità del dataset e la ricchezza delle informazioni.
 - Questo tipo di commit consente di evitare l'introduzione di una quantità significativa di rumore negli studi MSR.
- L'inclusione di tutti i commit, compresi quelli al di fuori dei rami principali, può aumentare la *complessità* dell'analisi. Tuttavia, può anche *arricchire* il set di dati aggiungendo informazioni importanti alla cronologia del software e fornendo una visione più completa degli sforzi collaborativi, delle strategie di branching e dei thread di sviluppo concorrenti.

Data Extraction

- **Data Cleaning**
 - *Nei sistemi di sviluppo software e di controllo versione, i commit rumorosi si riferiscono a set di modifiche o commit considerati estranei all'obiettivo principale dell'analisi. In genere, questi includono modifiche minori come la correzione di errori di battitura, la modifica dei nomi delle attività o la risoluzione di avvisi di lint.*
- Pertanto, un ricercatore MSR dovrebbe seguire i seguenti passaggi per ottenere il dataset ottimale:
 - Identificare i criteri per la selezione dei commit chiave da includere nello studio.
 - Valutare l'inclusione o l'esclusione di commit complessi, di merge e di quick-remedy.
 - Sulla base dei criteri definiti, identificare ed escludere i commit rumorosi nei dati estratti.

Homework

- Applicare alcuni degli algoritmi dello studio di **Kamei et al.** per identificare i commit che introducono e risolvono i bug nei repository.
- Partendo da repository selezionati, identificare i commit che correggono i difetti.
- Durante questo processo, pulire meticolosamente i dati rimuovendo eventuali commit complessi e di merge.
- Estrarre il messaggio di commit e l'insieme dei file modificati da ciascun commit che corregge i difetti.

Code Analysis

- **Static Code Analysis**

- L'analisi statica del codice è utile per
 - (i) garantire che il codice rispetti le regole sulle buone pratiche di programmazione,
 - (ii) individuare difetti e problemi di sicurezza e
 - (iii) identificare "code smell" e casi di debito tecnico.
- L'analisi statica può essere applicata durante le prime fasi dell'implementazione del software per un'individuazione precoce dei difetti, poiché il codice non deve essere completamente funzionante o eseguibile.

Code Analysis

- **Static Code Analysis**

- Il codice sorgente può essere analizzato a diversi livelli di granularità (dipende dall'obiettivo dello studio e dalle domande di ricerca).
 - (i) livello di *metodo*: il livello di granularità più basso, che consente un'analisi più dettagliata;
 - (ii) livello di *classe*: un livello di granularità più grossolano rispetto al livello di metodo, ma comunque diffuso;
 - (iii) livello di *file*: un livello di granularità più elevato e un livello di dettaglio inferiore quando si analizzano gruppi di classi;
 - (iv) livello di *sistema*: il livello di granularità più alto e il livello di dettaglio più basso.

Code Analysis

- **Static Code Analysis**

- Alcuni tool per analisi statica automatica

- PMD supporta 16 linguaggi, ma si concentra principalmente su Java. Ad esempio, tramite le regole di Java:
 - problemi di progettazione,
 - costrutti che sono difettosi, estremamente confusi o soggetti a errori di runtime,
 - problemi di documentazione del codice, codice non ottimale, potenziali falle di sicurezza e problemi nella gestione di più thread di esecuzione.
 - SpotBugs (precedentemente FindBugs) si focalizza principalmente sul rilevamento dei difetti basato su un set predefinito di oltre 400 modelli di bug nel codice Java.
 - la presenza di cattive pratiche, la correttezza, le prestazioni, il codice dannoso e i problemi di sicurezza.
 - Fortify specifico per le violazioni delle regole e delle linee guida di codifica specifiche per la sicurezza in diversi linguaggi.
 - analizzatori di vulnerabilità: buffer, configurazione, contenuto, flusso di controllo, flusso di dati, puntatore nullo, semantico e strutturale.

Code Analysis

- **Dynamic Program Analysis**

- A differenza dell'analisi statica, l'analisi dinamica mira a identificare i problemi durante l'esecuzione del codice.
- L'analisi dinamica è utile per identificare
 - (i) una scarsa copertura del codice
 - (ii) allocazioni e perdite di memoria
 - (iii) colli di bottiglia nelle prestazioni
 - (iv) vulnerabilità e difetti del software
 - (v) problemi di concorrenza come deadlock e race condition.

Code Analysis

- **Dynamic Program Analysis**

- Alcuni strumenti di analisi dinamica degni di nota includono:
 - Valgrind fornisce una suite di strumenti per la creazione di strumenti di analisi dinamica.
 - Memcheck può rilevare molti errori relativi alla memoria, comuni nei programmi C e C++, che possono causare arresti anomali e comportamenti imprevedibili;
 - Helgrind è un debugger di thread che individua le race condition nei programmi multithread;
 - Massif esegue una profilazione dettagliata dell'heap, individuando le parti del programma responsabili della maggior parte dell'allocazione di memoria.
 - AppVerifier di Microsoft è utile per rilevare e facilitare il debug di corruzioni della memoria, vulnerabilità di sicurezza critiche e problemi relativi ai privilegi limitati degli account utente, che sarebbero difficili da individuare durante i normali test delle applicazioni.
 - Code Pulse è uno strumento di copertura del codice in tempo reale per le attività di penetration testing di OWASP e Code Dx (acquisita da Synopsys) che rileva automaticamente le informazioni sulla copertura del codice durante l'esecuzione dei test. Supporta programmi Java e .NET Framework.

Homework

- Recuperare le versioni del codice da un repository specifico sia prima che dopo un commit di correzione di un difetto.
- Utilizzando uno strumento di analisi a scelta, come SpotBugs, esplorare le implicazioni delle correzioni dei difetti in termini di correttezza, prestazioni e sicurezza.

Mined Data Analysis

- L'analisi dei dati raccolti dipende principalmente dal tipo di dati, ma anche dallo scopo dell'analisi e dagli strumenti e le tecniche disponibili.
- Molteplici fonti e tipologie di dati: codice sorgente, commit, problemi di GitHub, domande e risposte, commenti di revisione, commenti al codice, dati numerici, ecc
- Approcci basati sulla modellazione degli argomenti e sul deep learning sono ampiamente utilizzati per l'analisi del codice sorgente.

Mined Data Analysis

- **Unstructured data**

- Possiamo utilizzare diverse tecniche per analizzare i dati testuali.
 - Il fattore principale che determina la tecnica di analisi da utilizzare è "cosa" intendiamo identificare nel nostro studio.
 - Tecniche più semplici, come l'analisi della frequenza delle parole basata sulle metriche TF-IDF (Term-Frequency Inverse-Document-Frequency),
 - Tecniche più approfondite per *comprendere il contenuto* dei documenti o categorizzarli, come
 - la modellazione degli argomenti,
 - il clustering e le tecniche di classificazione,
 - Analisi sofisticate come: la *ricerca del razionale, relazioni o causalità tra i concetti* (tramite inferenza causale).
- In tutti i casi, i dati testuali possono essere rappresentati con vettori numerici che possono essere appresi utilizzando *deep neural models*.

Mined Data Analysis

- **Unstructured data - Topic Modeling**
 - Il corpus di documenti può essere costituito da testi brevi o lunghi, come messaggi di commit, feedback di revisione di app o segnalazioni di bug.
 - ogni commit, revisione di app o segnalazione di bug è considerato un documento.
 - Un *topic* è una raccolta di termini che ricorrono frequentemente nei documenti.
 - Viene utilizzata per diverse applicazioni di ingegneria del software, come la classificazione dei bug, l'individuazione degli argomenti o dei problemi più discussi, il recupero dei link di tracciabilità e l'individuazione dei concetti.
 - Esempi di algoritmi: Latent Semantic Indexing (LSI), Probabilistic Latent Semantic Indexing (PLSI), Latent Dirichlet Allocation (LDA) and its variations, etc....
 - Gli algoritmi più recenti si basano su word embedding o su modelli di argomenti basati su reti neurali, come Top2Vec, LDA2Vec e BERTopic.

Mined Data Analysis

- **Unstructured data - Topic Modeling**

- Considerazioni:

- Text pre-processing: *e.g., rimuovi la punteggiatura e i simboli non testuali*
 - Investigating the length of the text: *la modellazione degli argomenti non funziona correttamente per testi brevi*
 - The evaluation metrics: qualità, interpretabilità, stabilità, diversità, efficienza e flessibilità.
 - Due termini ben noti per misurare i modelli tematici sono *perplexità* e *coerenza*. Il primo si riferisce a quanto bene il modello spiega i dati (potere predittivo del modello), mentre il secondo identifica una misura della possibilità di associare un unico significato semantico alle parole generate per un determinato argomento.
 - Number of topics: *evitare di utilizzare numeri casuali, sperimentare scelte diverse*
 - Hyperparameter setting: *sperimentare scelte diverse*
 - Running time and size of data: *problema di scalabilità*

Mined Data Analysis

- **Unstructured data - Sentiment analysis**

- L'analisi del sentiment, detta anche opinion mining, si riferisce alla raccolta e all'analisi delle opinioni, delle emozioni o degli atteggiamenti delle persone nei confronti di un'entità (ad esempio, individui, prodotti, argomenti, ecc.).
- Nell'ingegneria del software, l'analisi del sentiment può analizzare diversi artefatti MSR, come *recensioni* di app, *feedback* degli utenti sui prodotti e *discussioni* tra sviluppatori.
- L'analisi del sentiment determina se l'opinione è positiva, negativa o neutra.
- L'analisi può essere eseguita a diversi livelli di granularità: documento, frase, espressione o persino determinati aspetti di un'entità.
- Sviluppati diversi algoritmi per l'analisi del sentiment, ad esempio:
 - approcci basati su lessico
 - tecniche di apprendimento automatico supervisionato e non supervisionato, inclusi modelli di deep learning e modelli linguistici pre-addestrati

Mined Data Analysis

- **Unstructured data – (Deep) Neural Network Based.**
 - Rappresentazione del documento tramite un vettore, chiamato embedding
 - può essere una rappresentazione testuale non contestuale o contestuale.
 - Per le rappresentazioni non contestuali, il modello apprende un vettore fisso per ogni parola, indipendentemente dal contesto. Esempi famosi di tali embedding sono word2vec, glove e code2vec.
 - Nella rappresentazione contestuale, l'embedding di una parola dipende dalle parole che la circondano.
 - Di recente, l'embedding del testo viene estratto da un modello linguistico, i.e., un modello che apprende un modello probabilistico di un linguaggio naturale da un ampio corpus di dati testuali.
 - È possibile utilizzare più modelli per estrarre gli embedding di testo o codice, inclusi i Transformer o la famiglia GPT per modelli linguistici specifici per testo o codice come CodeBERT, GraphCodeBERT, CodeT5 o Code Llama.

Mined Data Analysis

- **Structured data**

- I dati strutturati vengono analizzati attraverso vari metodi, come l'analisi di correlazione, l'analisi delle serie temporali, l'analisi statistica, il clustering, l'analisi di regressione e lo sviluppo di modelli predittivi.
- I test parametrici e non parametrici sono classi specifiche che si riferiscono alla distribuzione dei parametri della popolazione.
 - Il t-test, l'ANOVA, e il test F sono alcuni test parametrici comuni.
 - Test non parametrici sono il test U di Mann-Whitney e le sue varianti, il test di Kruskal-Wallis
- Statistical power:
- Effect size:

Code Analysis

- **Data Synthesis**

- Seguendo l'approccio GQM (presentato nella Sezione 2.1), tutti i grafici e i risultati mostrati nello studio mirano a rispondere alle domande di ricerca formulate.
- Obiettivi della ricerca come guida: *considerare link tra la domanda di ricerca e i risultati*
- Rilevanza per le domande di ricerca: *mantenere una narrazione coerente rispetto allo scopo dello studio.*
- Gestione di risultati inattesi: *potrebbe far emergere nuove implicazioni e aprire la possibilità di nuove opportunità.*
- Metriche descrittive: *utili per distillare insieme di dati complessi e ricavarne informazioni significative e interpretabili.*
- Rappresentazioni grafiche:
 - Grafici a linee:
 - Grafici a barre:
 - Box plot:
 - Grafici a dispersione:
 - Grafici di rete:

Threats to Validity

- Conclusion validity
 - minacce legate alle analisi dei risultati statistici (ad esempio, bassa potenza statistica e violazione delle ipotesi del test statistico),
 - le minacce specifiche nei processi MSR riguardano fattori confondenti:
 - *affidabilità delle misure* (soprattutto se vengono utilizzate analisi dinamiche del codice),
 - *irrilevanze casuali nel processo di data mining* (ad esempio, periodi prolungati di inattività del repository) e
 - *eccessiva eterogeneità dei repository* (ad esempio, considerando nello stesso processo MSR contesti sia cloud-native che embedded).
 - Le strategie di mitigazione prevedono la selezione dei repository sulla base di criteri definiti a priori, l'uso ponderato di strumenti di analisi del codice sorgente affidabili e un esame a posteriori delle tendenze dei risultati per individuare potenziali insidie nelle conclusioni relative a repository specifici.

Threats to Validity

- Internal validity
 - spesso correlate a una selezione sommaria del repository, a un'elaborazione successiva imprecisa dei dati estratti, ad analisi statistiche dei dati non adeguate e, quando si utilizzano analisi dinamiche, all'influenza delle precedenti esecuzioni del codice su quelle successive.
 - possono spesso essere affrontate durante la fase di progettazione dell'estrazione dei dati, garantendo che venga utilizzato un processo sistematico, ripetibile e documentato per selezionare i repository e gestire i dati raccolti, e che vengano adottate precauzioni di "cooldown" e pulizia della cache per avviare ogni misurazione dinamica come se fosse una tabula rasa.

Threats to Validity

- Construct validity
 - nei processi MSR sono legate alla fase di progettazione dei processi di mining e possono comportare:
 - *una definizione vaga o inadeguata del costrutto* (ad esempio, definire il debito tecnico come problemi di manutenibilità),
 - *una sottorappresentazione* del costrutto considerato (ad esempio, considerare un singolo tipo di instabilità dei test per studiare l'argomento)
 - l'adozione di *un'analisi eccessivamente ristretta* per esaminare un costrutto considerevolmente più ampio (ad esempio, concentrarsi esclusivamente sul numero di funzioni per studiare la testabilità del software).
 - Durante la progettazione dei processi MSR, il costrutto oggetto di studio e il modo in cui il mining considera tale costrutto dovrebbero essere attentamente discussi tra i partecipanti al mining per comprendere, mitigare e documentare i potenziali rischi di test del costrutto implicati dalla metodologia adottata.

Threats to Validity

- External validity
 - Una delle più comuni nell'MSR riguarda la *sottorappresentazione dell'intero insieme di repository software e commit rilevanti* per rispondere alle domande di ricerca.
 - Una strategia per campionare sistematicamente repository e commit è fondamentale per mitigare questa minaccia, considerando il rischio che tale campionamento comporta.
 - Un campionamento valido dovrebbe selezionare attentamente sia una tecnica di campionamento appropriata (ad esempio, campionamento stratificato se è necessario rappresentare repository di diversa natura) sia una dimensione del campione.
 - Per quanto riguarda la dimensione del campione, è possibile utilizzare numerosi strumenti per calcolarla in base all'intervallo di confidenza e al margine di errore, tipicamente rispettivamente del 95% e del 5%.
 - Un altro comune rischio è l'adozione di processi ad hoc, manuali o obsoleti per l'estrazione e l'analisi del codice sorgente.
 - Utilizzare strumenti di analisi e repository affidabili
 - è opportuno considerare altre tecniche di campionamento, come il campionamento stratificato.

Complementing Software Repository Mining Studies

- **... with Qualitative Methods**
 - Ad esempio, integrare un'esplorazione MSR con metodi di ricerca qualitativa, come sondaggi, studi basati su interviste e focus group.
 - Survey: potrebbe cogliere le esperienze e le percezioni soggettive di sviluppatori, project manager e altri stakeholder, fornendo una narrazione umana a complemento delle tendenze quantitative identificate nell'esplorazione MSR.
 - Un'intervista è un metodo di ricerca qualitativa che raccoglie e analizza sistematicamente dati da interviste individuali con i partecipanti. A differenza degli studi basati su sondaggi, le interviste forniscono ai ricercatori approfondimenti più dettagliati sulle esperienze degli intervistati.
 - possono raggiungere solo un campione di piccole dimensioni e sono in genere limitati all'analisi di pochi professionisti.
 - particolarmente adatti quando l'obiettivo è ottenere una comprensione approfondita e sfumata delle esperienze, delle prospettive o delle convinzioni dei partecipanti.
 - I focus group: un metodo di ricerca qualitativo che coinvolge un piccolo gruppo eterogeneo di partecipanti che si impegnano in una discussione aperta e facilitata su un argomento specifico sotto la guida di un moderatore.

Complementing Software Repository Mining Studies

- **Key Advantages of Mixed-Method Research**
 - Mentre gli studi MSR forniscono la base di prove empiriche, i metodi qualitativi infondono una dimensione umana, svelando le storie dietro le modifiche al codice.
 - Approfondire i sistemi di tracciamento dei problemi, i repository di revisione del codice e i forum degli sviluppatori aggiunge livelli di contesto, rappresentando lo sviluppo del software come un percorso collaborativo e in continua evoluzione piuttosto che una semplice raccolta di modifiche al codice.
 - In sostanza, questa integrazione trascende i confini delle metodologie quantitative e qualitative, promuovendo un approccio di ricerca che rispecchia la complessità dell'ecosistema di sviluppo del software.

Ethical Mining

- Quando si analizzano i dati generati dall'uomo provenienti dai repository, alcune raccomandazioni (basate sul rapporto Menlo [GK22]) includono:
 - Identificazione delle parti interessate
 - Consenso informato
 - Bilanciamento dei rischi e dei benefici
 - Equità e imparzialità
 - Conformità, trasparenza e responsabilità

[GK22] Nicolas E Gold and Jens Krinke. *Ethics in the mining of software repositories*. Empirical Software Engineering, 27(1):17, 2022.

Ethical Mining

- Alcuni punti da considerare quando si conducono studi MSR includono:
 - Un cambio di mentalità rispetto al semplice "Ecco un dataset, vediamo cosa riusciamo a trovare", poiché questo può essere rischioso per i partecipanti.
 - Pertanto, le considerazioni etiche dovrebbero valutare le potenziali aree di danno, comprese le osservazioni e i giudizi, e valutare l'impatto che i risultati della ricerca possono avere sugli individui/sviluppatori.
- Gli studi MSR vengono spesso condotti su un campione di dati, che è casuale.
 - L'etica potrebbe svolgere un ruolo nel garantire che il campione rappresenti le persone coinvolte e sia inclusivo per quanto riguarda le domande e il processo. La considerazione etica garantisce che individui e analisi non siano esclusi dai risultati.
- Esistono diverse leggi e regolamenti in materia di privacy, tra cui:
 - il Regolamento generale sulla protezione dei dati (GDPR) dell'UE e il California Consumer Privacy Act del 2018 (CCPA), oltre alle leggi sulla proprietà intellettuale (IP).
 - Diverse licenze stabiliscono anche diverse autorizzazioni d'uso.
- Queste leggi a volte limitano usi e ricerche specifici e dovrebbero essere considerate nella progettazione dello studio o prima della raccolta dei dati.
 - questo aspetto dovrebbe essere considerato e discusso attivamente nell'articolo come una minaccia alla validità.

Conclusioni

- MSR è un approccio fondamentale in software engineering
- Permette analisi empiriche su grandi quantità di dati
- Può essere combinato con metodi qualitativi per risultati più completi
- ***Una tendenza di rilievo è la crescente enfasi sull'utilizzo delle tecniche di intelligenza artificiale (IA) per estrarre informazioni utili da vasti archivi di dati relativi al software.***